

Tipos Abstractos de Datos

Profesor: Pablo J Piccoli Bornia

Auxiliares: Daniela Longas, M Victoria Tolosa

1.
[Tipos Abstractos de Datos](#)
2.
[Introducción a diccionarios](#)
3.
[Operaciones básicas con diccionario](#)
[s](#)
4.
[Ejercicios](#)

- Mencionamos que una forma de **abordar un problema** es dividirlo en **subproblemas** más pequeños.
- Utilizar **funciones** nos permite generar un código más claro, que puede ser **reutilizado**.
- Las funciones también nos permiten aislar **unidades conceptuales de cómputo**.

Vamos a extender esta idea con el concepto de **Tipos (Abstractos) de Datos**
Por qué? modularidad, encapsulamiento.

Recordemos que Excel nos permite operar con *fechas*.

Cómo representa Excel las fechas?

De qué otra manera (eventualmente más simple) podríamos representarlas?

Qué información debería tener una fecha?

TIPOS ABSTRACTOS DE DATOS

Una fecha está definida por:

- ☐ Un número que represente el *día*.
- ☐ Un número que represente el *mes*.
- ☐ Un número que represente el *año*.

Dada una fecha, ¿qué nos gustaría preguntarle y/o pedirle?

Una fecha está definida por:

- ☐ Un número que represente el *día*.
- ☐ Un número que represente el *mes*.
- ☐ Un número que represente el *año*.

Dada una fecha, qué nos gustaría preguntarle y/o pedirle?

- ☐ Saber cuál es el día.
- ☐ Saber cuál es el mes.
- ☐ Saber cuál es el año.
- ☐ Que devuelva un string con la fecha en determinado formato (US, por ejemplo).

Sería interesante contar con un **tipo de datos** que *encapsule* una **fecha**.

Posibles *operaciones* del Tipo Abstracto de Datos **fecha**.

- CrearFecha(dia,mes,año), que crea una fecha con los parámetros int dia, mes y año.
- Dia(f), que dada una fecha f devuelve un int con el día.
- Mes(f), que dada una fecha f devuelve un int con el mes.
- Año(f), que dada una fecha f devuelve un int con el año.
- ImprimirFormatoUS(f), que dada una fecha f devuelve un string con la fecha en formato mm/dd/aaaa.

Definición

Un **Tipo Abstracto de Datos** (TAD) es un modelo matemático, lógico, que describe las operaciones y como manipular un **tipo de datos**.

Métodos Analíticos y Programación

○ Cuáles son los detalles de implementación de un TAD?

Al usuario (programador) **no** le interesa.

○ Qué necesitamos para poder interactuar con un TAD?

El **nombre** del tipo de datos, qué **operaciones** (funciones, procedimientos) provee, **restricciones**

Intuición

Un **TAD** se puede implementar de muchas maneras distintas, y es transparente para el **usuario** (léase, programador).

Estructura de datos

Una **estructura de datos** es una **implementación concreta** de un TAD. Es una *colección de valores relacionados*. La misma organiza la información de una particular, con una representación específica que relaciona los valores y provee operaciones y funciones que pueden aplicarse con sus respectivas garantías de eficiencia.

Retomemos el ejemplo del TAD
fecha.

**Alternativas para representar una
fecha?**

**Ventajas y
desventajas?**

**Qué estructuras de datos estuvimos
usando?**

Algunos tipos/estructuras de datos que vimos hasta ahora:

- ☐ int
- ☐ float
- ☐ string
- ☐ **list**

Algunas operaciones que vimos sobre list:

- ☐ l[i]: accede a la posición i de l.
- ☐ len(l): retorna la cantidad de elementos de l.
- ☐ l.append(elem): agrega elem al final de l. **Procedimiento.**
- ☐ l.pop(): quita el último elemento de l. **Procedimiento.**

Por qué es importante entender qué es un TAD, una estructura de datos, y la diferencia entre ellos?

Table of Contents

- 5. Data Structures
 - 5.1. More on Lists
 - 5.1.1. Using Lists as Stacks
 - 5.1.2. Using Lists as Queues
 - 5.1.3. List Comprehensions
 - 5.1.4. Nested List Comprehensions
 - 5.2. The `del` statement
 - 5.3. Tuples and Sequences
 - 5.4. Sets
 - 5.5. Dictionaries
 - 5.6. Looping Techniques
 - 5.7. More on Conditions
 - 5.8. Comparing Sequences and Other Types

5. Data Structures

This chapter describes some things you've learned about already in more detail, and adds some new things as well.

5.1. More on Lists

The list data type has some more methods. Here are all of the methods of list objects:

`list.append(x)`

Add an item to the end of the list. Equivalent to `a[len(a):] = [x]`.

`list.extend(iterable)`

Extend the list by appending all the items from the iterable. Equivalent to `a[len(a):] = iterable`.

`list.insert(i, x)`

Insert an item at a given position. The first argument is the index of the element before which to insert, so `a.insert(0, x)` inserts at the front of the list and `a.insert(len(a), x)` is equivalent to

Sabemos cómo está implementada list?

Las estructuras de datos cuentan generalmente con dos *partes*:

- **Parte pública.** Disponible al usuario externo. El nombre del tipo, las operaciones, funciones. Restricciones sobre su utilización. *Garantías de eficiencia (complejidad).*
- **Parte privada.** Sólo accesible dentro del tipo/estructura de datos. Contiene la implementación concreta del mismo, incluyendo su *representación interna* y la definición de los métodos (funciones y procedimientos) provistos por el mismo. **Utilizado por los programadores del tipo/estructura.**

INTRODUCCIÓN A DICCIONARIOS

Hasta ahora la estructura de datos más *compleja* con la que trabajamos es list. Una de sus principales características es que permite trabajar con una **colección de elementos**, que poseen un **orden** y se acceden mediante **índices numéricos**.

Diccionario

Un **diccionario** es un contenedor que permite asociar un **significado (valor)** a una determinada **clave**.

Intuición

Los **diccionarios** son similares a las listas en el sentido de que se utilizan para guardar elementos. En las listas, los *índices* deben ser números enteros. Para los diccionarios, pueden ser tipos de datos más generales.

Como se compone un diccionario:

- contiene una colección de *índices*, que se los denomina **claves**;
- contiene una colección de *valores*, que se los denomina **significados**;
- cada **clave** está asociada con un **único significado**. A cada par (**clave, significado**) se lo denomina **item**.

En ese caso, decimos que tenemos un `dict(string,string)` (y que se lee como *diccionario de string en string*).

Qué operaciones nos gustaría pedirle a un diccionario?

En ese caso, decimos que tenemos un `dict(string,string)` (y que se lee como *diccionario de string en string*).

Qué operaciones nos gustaría pedirle a un diccionario?

- ☐ Crear un diccionario.
- ☐ Definir una palabra (key) con su significado (value).
- ☐ Obtener el significado (value) de una palabra (key).
- ☐ Consultar si una palabra (key) está definida.
- ☐ Obtener el conjunto de palabras (keys).
- ☐ Obtener el conjunto de significados (values).
- ☐ Borrar una palabra (key) y su significado (value).

OPERACIONES BÁSICAS DE DICCIONARIOS

○ **Crear un diccionario.** Existen dos formas de crear un diccionario: usando

`{ }` o la función `dict()`.

```
>>> eng2sp = { }  
>>> eng2sp = dict ( )
```

El inicio/fin del diccionario se representa con llaves `{ }`. Los **ítems** se separan por comas (,), y el doble punto (:) relaciona clave con significado.

```
>>> eng2sp = { 'one': 'uno ', 'two': 'dos' }  
>>> print ( eng2sp )
```

Intuitivamente, gran parte de las operaciones se especifican de manera similar a las listas. Sin embargo, el contenedor es más complejo (e internamente están sucediendo otras cosas).

- Acceso/definición/modificación de elementos. Todas estas tareas se realizan utilizando el operador [].

```
>>> eng2sp = { 'one': 'uno', 'two': 'dos' }  
>>> uno = eng2sp [ 'one' ] # Pedimos significado de clave 'uno'  
>>> print ( uno )  
>>> otro_val = eng2sp [ 'tres' ] # Que sucede?
```

Podemos definir nuevos ítems y modificar la definición de alguno ya existente.

```
>>> eng2sp = { 'one': 'uno', 'two': 'dos' }  
>>> eng2sp [ 'three' ] = 'tres'  
>>> eng2sp [ 'two' ] = 'tres'
```

Para saber si una clave se encuentra definida o no en el diccionario podemos usar el operador in

```
>>> eng2sp = { 'one': 'uno', 'two': 'dos' }  
>>> 'one' in eng2sp  
True  
>>> 'four' in eng2sp  
False
```

- **Eliminar una clave y su significado.** De forma similar a la lista, podemos utilizar la función `del`.

```
>>> eng2sp = { 'one': 'uno', 'two': 'dos' }  
>>> del eng2sp [ 'one' ]  
>>> eng2sp  
{ 'two': 'dos' }
```

- **Cantidad de ítems.** La función `len` nos permite saber cuántos ítems tiene un diccionario.

```
>>> eng2sp = { 'one': 'uno', 'two': 'dos' }  
>>> len ( eng2sp )  
2
```

Cualquier tipo de dato puede ser clave y/o significado?

Cómo recorreremos un diccionario?

Qué reglas deben seguir los conjuntos de claves y significados?

Algunas buenas prácticas

EJERCICIOS

Tenemos el siguiente diccionario guardado en una variable que indica para cuatro personas cuál es su signo en el horóscopo chino:

```
horosocopo_chino = { 'Jaime': 'perro', 'Mariana': 'chancho', 'Juan': 'gallo', 'Ana': 'perro' }
```

En función de las operaciones vistas, determinar si es posible escribir en Python las siguientes consultas y, en caso afirmativo, hacerlo.

- ☐ Ana está entre las claves del diccionario.
- ☐ Jaime no está entre las claves del diccionario.
- ☐ El signo zodiacal de Mariana es **perro**.
- ☐ alguna persona tiene como signo zodiacal **pato**.